

Reviewer Pack — Algebraic Stochastic Order of Boolean Functions vs Resolution...

Entry ddff7b518beb · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-24 17:21:29 UTC

Algebraic Stochastic Order of Boolean Functions vs Resolution Proof Width

Entry ID: ddff7b518beb **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-24 17:21:29 UTC

1. Conjecture

Field A (mathematical branch): Stochastic Order Theory **Field B** (complexity object): Complexity Theory (Resolution Proof Complexity)

Statement:

[‘For a CNF formula F with n variables, let $\alpha(F)$ be the algebraic stochastic order between the boolean functions associated with F and its negation. Then for all instances F , the resolution proof width $t(F)$ satisfies: $t(F) \leq (4/3)^{\alpha(F)n}$.’]

Rationale (proposer’s reasoning):

[“Stochastic order theory provides a framework to compare the ‘stochastic behavior’ of different mathematical objects. By linking it with resolution proof complexity, we may uncover novel relationships between the structure of CNFs and their proof difficulty. This could potentially lead to new insights in proving lower bounds for NP-complete problems.”]

Taxonomy category: StochasticOrderToResolutionProofWidth (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 31b2ffc798e66a65

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the Spearman rank correlation coefficient between algebraic stochastic order (α) and logarithm of resolution proof width ($\log_2(t^*)$) for all 30 CNF formulas is significantly negative (p-value < 0.05).

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	1.00	UNCERTAIN	SAFE
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	0.90	HITS	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 6 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - intitle:(algebraic stochastic order OR α) AND title:(boolean functions OR CNF formula) - title:(resolution proof width) AND subject:(algebraic stochastic order OR α) - subject:(stochastic order theory) AND subject:(complexity theory) AND title:(resolution proof width)

Top relevant hits considered: - [http://arxiv.org/abs/1410.4915v3] Rankin-Selberg L-functions in cyclotomic towers, III - [http://arxiv.org/abs/1710.05169v1] Doubly Damped Stochastic Parallel Translations and Hessian Formulas - [http://arxiv.org/abs/1304.7603v1] Cylindrical Algebraic Decompositions for Boolean Combinations - [http://arxiv.org/abs/2412.20985v3] Eigenvalues of a third order BVP subject to functional BCs - [http://arxiv.org/abs/2204.08071v1] Eigen mode selection in human subject game experiment - [http://arxiv.org/abs/1303.0217v2] A stochastic diffusion process for the Dirichlet distribution

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def generate_cnf(n, m):
    cnf = []
    for _ in range(m):
        clause = [random.choice([-1, 1]) * (i + 1) for i in range(n)]
        if all(c == 0 for c in clause):
            clause[random.randint(0, n - 1)] = random.choice([-1, 1])
        cnf.append(clause)
    return cnf

def algebraic_stochastic_order(cnf):
    n = len(cnf[0])
    assignments = [tuple(random.choices([0, 1], k=n)) for _ in range(2**n)]

    def evaluate_cnf(assignments, cnf):
        results = []
        for assignment in assignments:
            result = 1
            for clause in cnf:
                if all((assignment[i-1] == 0 and c > 0) or (assignment[i-1] == 1 and c < 0) for i, c in enumerate(clause)):
                    result *= -1
            results.append(result)
```

```

        return results

results_pos = evaluate_cnf(assignments, cnf)
results_neg = evaluate_cnf([(1 - a) for a in assignment] for assignment in assignments], cnf)

alpha_F = sum([results_pos[i] * results_neg[i] for i in range(len(results_pos))]) / len(results_pos)
return alpha_F

def resolution_width(cnf):
    n = len(cnf[0])
    clauses = {tuple clause for clause in cnf}
    queue = list(clauses)
    literals_seen = set()

    while queue:
        literal = random.choice(list(literals_seen))
        new_clauses = []
        for clause in queue:
            if literal in clause:
                continue
            if -literal in clause:
                return len(queue)
            new_clause = [l for l in clause if l != -literal]
            if new_clause and tuple(new_clause) not in clauses:
                new_clauses.append(tuple(new_clause))
        queue.extend(new_clauses)
        literals_seen.add(literal)

    return len(queue)

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = random.choice([5, 10, 15, 20, 30, 40])
    m = random.randint(2 * n, 3 * n)
    cnf = generate_cnf(n, m)

    alpha_F = algebraic_stochastic_order(cnf)
    t_F = resolution_width(cnf)

    return {
        "metric_name": "algebraic_stochastic_order",
        "metric_value": alpha_F,
        "instances_tested": 1,
        "conjecture_holds": t_F <= (4/3)**alpha_F * n,
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i + 1 for i in range(5, 8)]

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

```

```

if all(result["conjecture_holds"] for result in results):
    mean_value = sum(result["metric_value"] for result in results) / len(results)
    support_fraction = 1.0
    print(f"RESULT: SUPPORTED mean={mean_value} std=0.0 support_fraction={support_fraction}")
else:
    first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
    counterexample = "mapping_undefined"
    print(f"RESULT: FALSEIFIED counterexample=\"{counterexample}\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_1c0621d1.py", line 91, in <module>
    result = run_trial(seed)
              ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_1c0621d1.py", line 59, in run_trial
    alpha_F = algebraic_stochastic_order(cnf)
              ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_1c0621d1.py", line 46, in algebraic_stochastic_order
    value = sum([assignment[i-1] * clause[i-1] for clause in cnf])
              ^
NameError: name 'i' is not defined. Did you mean: 'id'?

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed during execution due to an undefined variable 'i', which prevented the production of data necessary to evaluate the conjecture. | next: Review and correct the error in the test code that caused it to crash, then rerun the test to gather the required data for evaluating the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	11251
2	propose	ollama_remote	glm4:latest	0	0	9375
3	preregistration	ollama_remote	glm4:latest	0	0	5419

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
4	novelty	ollama_remote	glm4:latest	0	0	4942
5	novelty	ollama_remote	glm4:latest	0	0	14389
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17221
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13758
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13410
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10303
10	judge	ollama_remote	glm4:latest	0	0	13305

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 113372 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/ddff7b518beb.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/ddff7b518beb.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/ddff7b518beb.tar.gz` (if generated)